12. Testing, Table Tests, Fuzz, and Benchmarks

12. Testing, Table Tests, Fuzz, and Benchmarks

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `12-testing`

Learning objectives

- Write table-driven tests
- Use subtests with t.Run
- Introduce fuzz targets and benchmarks
- Treat tests as executable specifications

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Table-driven tests

```
func TestClamp(t *testing.T) {
    tests := []struct{
        name string
        v, lo, hi, want int
    }{
        {"mid", 5, 0, 10, 5},
        {"low", -1, 0, 10, 0},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            if got := Clamp(tt.v, tt.lo, tt.hi); got != tt.want {
                t.Fatalf("got %d want %d", got, tt.want)
            }
        })
    }
}
```

Fuzzing

```
func FuzzReverse(f *testing.F) {
    f.Add("go")
    f.Fuzz(func(t *testing.T, s string) {
        // property: reverse(reverse(s)) == s for runes...
    })
}
```

Benchmarks

```
func BenchmarkSum(b *testing.B) {
    xs := make([]int, 1000)
```

```
    for i := 0; i < b.N; i++ {
        Sum(xs)
    }
}
```

Run: `go test -bench=. -benchmem ./...`

Common mistakes (especially from Python instincts)

- Giant tests without cases (hard to see failures)
- Testing implementation details instead of behavior
- Ignoring flaky time/network tests — inject clocks and interfaces

Exercises

Work in `exercises/12-testing/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/12-testing/` until you have an honest failing attempt.

Self-check

1. Why table tests scale better?
2. What does b.N mean?
3. When is fuzzing valuable?

Further reading

- https://go.dev/doc/tutorial/add-a-test
- https://go.dev/security/fuzz/